TAREA – IMPLEMENTACIÓN DE LA ESTRUCTURA COMPLETA DE UNA BASE DE DATOS (DDL)

Bases de Datos I

Juan David Jovel López
Angie Vanessa Rincón Osorio

Programa: Tecnología en desarrollo de software Virtual

Grupo: G100

Docente: Oralia Cortés Grajales

Institución: Institución Universitaria Pascual Bravo

Fecha de entrega: 18 de octubre de 2026

Link GitHub: https://github.com/juanjovel652-hub/bd1-redsocial-grupo3

Link video: https://youtu.be/ygjrELhACM0

Informe técnico: modelo físico de la red social estudiantil pascualina

1.Identificación del SGBD

El sistema gestor utilizado fue **PostgreSQL**, indicado por la profesora para el desarrollo del proyecto. A modo de comparación se consideraron también MySQL y SQLite.

Criterio	PostgreSQL	MySQL	SQLite
Arquitectura	Cliente-servidor	Cliente-servidor	Embebido, un solo archivo, sin servidor
Cumplimiento del estándar SQL	Muy alto	Alto, con particularidades propias	Parcial
Integridad referencial (PK, FK, CHECK)	Completa; CHECK y acciones ON UPDATE/ON DELETE plenamente aplicadas	Completa en InnoDB; CHECK solo desde la versión 8.0.16	Soporta FK, pero deben activarse con PRAGMA foreign_keys = ON
Columnas autoincrementales	GENERATED ... AS IDENTITY (estándar SQL)	AUTO_INCREMENT (propio)	AUTOINCREMENT (limitado)
Tipos de datos	Muy amplio (TEXT, TIMESTAMP, JSONB, arreglos, etc.)	Amplio	Tipado dinámico y flexible
Concurrencia y volumen	Muy buena (MVCC)	Buena	Limitada (un solo escritor)
Herramienta gráfica	pgAdmin 4	MySQL Workbench	DB Browser for SQLite

Ventajas de PostgreSQL para este proyecto

- Respeta de forma estricta tipos, restricciones y claves foráneas, lo que da coherencia al modelo de 15 tablas.

- Permite columnas de identidad estándar (GENERATED BY DEFAULT AS IDENTITY) y restricciones CHECK (usada en seguir).
- Maneja bien la concurrencia de una red social con muchos usuarios simultáneos.
- Es de código abierto y cuenta con pgAdmin 4 para ejecutar el script y generar el diagrama.

Desventajas de PostgreSQL

- Requiere instalación y configuración de un servidor, más pesado que SQLite.
- Su administración tiene una curva de aprendizaje mayor que la de MySQL.

MySQL: es muy difundido y fácil de administrar, pero su sintaxis tiene variaciones propias respecto al estándar y su manejo histórico de CHECK fue más débil.

SQLite: es ideal para prototipos y aplicaciones locales por no necesitar servidor, pero no se adapta a una red social con múltiples usuarios concurrentes y su control de integridad es menos estricto por defecto.

2.Transformación del modelo lógico al físico

Las reglas aplicadas para pasar del diagrama lógico al esquema SQL fueron:

- **Entidades → tablas.** Cada entidad se convirtió en una tabla: estudiante, perfil, habilidad, interes, mensaje, grupo, evento, publicacion, comentario y reaccion.
- **Atributos → columnas.** A cada atributo se le asignó un tipo de dato y, cuando correspondía, un tamaño (VARCHAR(n)). Los textos largos (biografía, descripciones, contenido) usan TEXT; las fechas con hora usan TIMESTAMP y las fechas simples DATE.
- **Identificadores → claves primarias.** Cada entidad tiene una PK surrogate de tipo INTEGER GENERATED BY DEFAULT AS IDENTITY, que genera el valor automáticamente.
- **Relaciones 1:1 → clave foránea con UNIQUE.** Estudiante–Perfil: perfil.id_estudiante es FK y además UNIQUE, de modo que un estudiante tiene como máximo un perfil.

- **Relaciones 1:N → clave foránea en el lado «muchos».** Grupo–Evento (evento.id_grupo), Estudiante–Publicacion, Publicacion–Comentario, Publicacion–Reaccion, Estudiante–Comentario y Estudiante–Reaccion.
- **Relaciones N:M → tabla intermedia con PK compuesta.** Se crearon estudiante_habilidad, estudiante_interes, miembro_grupo y asistente_evento. La PK compuesta evita duplicados (un estudiante no puede registrar dos veces la misma habilidad, interés, pertenencia a un grupo o asistencia a un evento). Los atributos propios de la relación (nivel, rol, estado, fechas) se guardaron en estas tablas.
- **Relaciones reflexivas (Estudiante–Estudiante).** seguir relaciona a un estudiante con otro (seguidor y seguido) con PK compuesta y un CHECK (id_seguidor <> id_seguido) para impedir que alguien se siga a sí mismo. mensaje tiene dos FK hacia estudiante (emisor y receptor).
- **Restricciones de dominio.** Se usó NOT NULL en los datos obligatorios, UNIQUE en los valores que no deben repetirse (correo institucional, nombre de habilidad, nombre de interés) y DEFAULT para valores iniciales (fecha actual, estado 'enviado', rol 'miembro', estado 'inscrito', tipo 'texto').

3.Descripción de las 15 tablas

estudiante

Datos principales de cada estudiante de la red. **PK:** id_estudiante.

Campo	Tipo	Restricciones	Justificación
id_estudiante	INTEGER (identity)	PK	Identificador automático y único
nombre	VARCHAR(80)	NOT NULL	Dato obligatorio de identificación
apellido	VARCHAR(100)	NOT NULL	Admite apellidos compuestos
correo_institucional	VARCHAR(150)	NOT NULL, UNIQUE	Garantiza una sola cuenta por correo

Campo	Tipo	Restricciones	Justificación
programa_academico	VARCHAR(120)	NOT NULL	Permite conectar estudiantes de la misma carrera
semestre	SMALLINT	NOT NULL	Número pequeño; permite identificar estudiantes de cursos avanzados para mentoría
fecha_registro	TIMESTAMP	NOT NULL, DEFAULT CURRENT_TIMESTAMP	Se completa automáticamente al registrarse

perfil

Información personal pública de un estudiante. **PK:** id_perfil. **FK:** id_estudiante.

Campo	Tipo	Restricciones	Justificación
id_perfil	INTEGER (identity)	PK	Identificador del perfil
id_estudiante	INTEGER	NOT NULL, UNIQUE, FK	Relación 1:1: un perfil por estudiante
biografia	TEXT	Opcional	Longitud variable y no obligatoria
ciudad	VARCHAR(100)	Opcional	Dato complementario
foto_perfil	TEXT	Opcional	Almacena la ruta o URL de la imagen, no el archivo
fecha_actualizacion	TIMESTAMP	NOT NULL, DEFAULT CURRENT_TIMESTAMP	Registra la última modificación

habilidad

Catálogo de habilidades (disciplinas, deportes, tecnologías). **PK:** id_habilidad.

Campo	Tipo	Restricciones	Justificación
id_habilidad	INTEGER (identity)	PK	Identificador
nombre	VARCHAR(100)	NOT NULL, UNIQUE	Evita habilidades repetidas en el catálogo
descripcion	TEXT	Opcional	Explicación libre

estudiante_habilidad

Tabla intermedia N:M entre estudiante y habilidad. **PK compuesta:** (id_estudiante, id_habilidad).

Campo	Tipo	Restricciones	Justificación
id_estudiante	INTEGER	PK, FK	Estudiante que posee la habilidad
id_habilidad	INTEGER	PK, FK	Habilidad asociada
nivel	VARCHAR(30)	Opcional	Nivel de dominio (ej. básico, intermedio, avanzado)
fecha_aprendizaje	DATE	Opcional	Solo importa el día, no la hora

interes

Catálogo de intereses de la comunidad. **PK:** id_interes.

Campo	Tipo	Restricciones	Justificación
id_interes	INTEGER (identity)	PK	Identificador
nombre	VARCHAR(100)	NOT NULL, UNIQUE	Evita intereses duplicados
descripcion	TEXT	Opcional	Explicación libre

estudiante_interes

Tabla intermedia N:M entre estudiante e interés. **PK compuesta:** (id_estudiante, id_interes).

Campo	Tipo	Restricciones	Justificación
id_estudiante	INTEGER	PK, FK	Estudiante que tiene el interés
id_interes	INTEGER	PK, FK	Interés asociado
fecha_agregado	TIMESTAMP	NOT NULL, DEFAULT CURRENT_TIMESTAMP	Momento en que se agregó

seguir

Relación reflexiva de seguimiento entre estudiantes. **PK compuesta:** (id_seguidor, id_seguido).

Campo	Tipo	Restricciones	Justificación
id_seguidor	INTEGER	PK, FK	Estudiante que sigue
id_seguido	INTEGER	PK, FK	Estudiante seguido
fecha_seguimiento	TIMESTAMP	NOT NULL, DEFAULT CURRENT_TIMESTAMP	Momento del seguimiento

Se agregó CHECK (id_seguidor <> id_seguido) para impedir el auto-seguimiento. La PK compuesta evita seguir dos veces a la misma persona.

mensaje

Mensajes directos entre estudiantes. **PK:** id_mensaje. **FK:** id_emisor, id_receptor.

Campo	Tipo	Restricciones	Justificación
id_mensaje	INTEGER (identity)	PK	Identificador
id_emisor	INTEGER	NOT NULL, FK	Quien envía
id_receptor	INTEGER	NOT NULL, FK	Quien recibe
texto	TEXT	NOT NULL	Longitud variable; un mensaje no puede ir vacío
estado	VARCHAR(20)	NOT NULL, DEFAULT 'enviado'	Seguimiento del mensaje (enviado, leído, etc.)

Campo	Tipo	Restricciones	Justificación
fecha_envio	TIMESTAMP	NOT NULL, DEFAULT CURRENT_TIMESTAMP	Fecha y hora automática

grupo

Grupos de estudio, equipos o clubes. **PK:** id_grupo.

Campo	Tipo	Restricciones	Justificación
id_grupo	INTEGER (identity)	PK	Identificador
nombre	VARCHAR(120)	NOT NULL	Todo grupo debe tener nombre
tema	VARCHAR(100)	Opcional	Materia o área del grupo
descripcion	TEXT	Opcional	Descripción libre

miembro_grupo

Tabla intermedia N:M entre estudiante y grupo. **PK compuesta:** (id_estudiante, id_grupo).

Campo	Tipo	Restricciones	Justificación
id_estudiante	INTEGER	PK, FK	Estudiante miembro
id_grupo	INTEGER	PK, FK	Grupo al que pertenece
rol	VARCHAR(30)	NOT NULL, DEFAULT 'miembro'	Distingue administradores de miembros
fecha_registro	TIMESTAMP	NOT NULL, DEFAULT CURRENT_TIMESTAMP	Fecha de ingreso

evento

Eventos creados por un grupo. **PK:** id_evento. **FK:** id_grupo.

Campo	Tipo	Restricciones	Justificación
id_evento	INTEGER (identity)	PK	Identificador

Campo	Tipo	Restricciones	Justificación
id_grupo	INTEGER	NOT NULL, FK	Grupo organizador (un evento pertenece a un solo grupo)
nombre	VARCHAR(150)	NOT NULL	Título del evento
fecha	TIMESTAMP	NOT NULL	Requiere fecha y hora de realización
lugar	VARCHAR(200)	Opcional	Puede ser un salón o un enlace virtual
descripcion	TEXT	Opcional	Detalle del evento

asistente_evento

Tabla intermedia N:M entre estudiante y evento. **PK compuesta:** (id_estudiante, id_evento).

Campo	Tipo	Restricciones	Justificación
id_estudiante	INTEGER	PK, FK	Estudiante inscrito
id_evento	INTEGER	PK, FK	Evento al que asiste
estado	VARCHAR(20)	NOT NULL, DEFAULT 'inscrito'	Estado de la asistencia (inscrito, asistió, etc.)
fecha_registro	TIMESTAMP	NOT NULL, DEFAULT CURRENT_TIMESTAMP	Momento de la inscripción

publicacion

Publicaciones de los estudiantes. **PK:** id_publicacion. **FK:** id_estudiante.

Campo	Tipo	Restricciones	Justificación
id_publicacion	INTEGER (identity)	PK	Identificador
id_estudiante	INTEGER	NOT NULL, FK	Autor de la publicación
contenido	TEXT	NOT NULL	Longitud variable

Campo	Tipo	Restricciones	Justificación
fecha	TIMESTAMP	NOT NULL, DEFAULT CURRENT_TIMESTAMP	Fecha y hora automática
tipo	VARCHAR(30)	NOT NULL, DEFAULT 'texto'	Clasifica la publicación (texto, recurso, noticia, etc.)

comentario

Comentarios sobre las publicaciones. **PK:** id_comentario. **FK:** id_publicacion, id_estudiante.

Campo	Tipo	Restricciones	Justificación
id_comentario	INTEGER (identity)	PK	Identificador
id_publicacion	INTEGER	NOT NULL, FK	Publicación comentada
id_estudiante	INTEGER	NOT NULL, FK	Autor del comentario
contenido	TEXT	NOT NULL	Un comentario no puede estar vacío
fecha	TIMESTAMP	NOT NULL, DEFAULT CURRENT_TIMESTAMP	Fecha y hora automática

reaccion

Reacciones a las publicaciones. **PK:** id_reaccion. **FK:** id_publicacion, id_estudiante.

Campo	Tipo	Restricciones	Justificación
id_reaccion	INTEGER (identity)	PK	Identificador
id_publicacion	INTEGER	NOT NULL, FK	Publicación que recibe la reacción
id_estudiante	INTEGER	NOT NULL, FK	Estudiante que reacciona
tipo_reaccion	VARCHAR(30)	NOT NULL	Tipo de reacción (me gusta, me encanta, etc.)

Campo	Tipo	Restricciones	Justificación
fecha	TIMESTAMP	NOT NULL, DEFAULT CURRENT_TIMESTAMP	Fecha y hora automática

4. Relaciones y acciones referenciales

Todas las claves foráneas se definieron con `ON UPDATE CASCADE ON DELETE CASCADE`.

Tabla padre	Tabla hija	Clave foránea	Cardinalidad	Acción
estudiante	perfil	id_estudiante	1:1	CASCADE / CASCADE
estudiante	estudiante_habilidad	id_estudiante	1:N	CASCADE / CASCADE
habilidad	estudiante_habilidad	id_habilidad	1:N	CASCADE / CASCADE
estudiante	estudiante_interes	id_estudiante	1:N	CASCADE / CASCADE
interes	estudiante_interes	id_interes	1:N	CASCADE / CASCADE
estudiante	seguir	id_seguidor	1:N	CASCADE / CASCADE
estudiante	seguir	id_seguido	1:N	CASCADE / CASCADE
estudiante	mensaje	id_emisor	1:N	CASCADE / CASCADE
estudiante	mensaje	id_receptor	1:N	CASCADE / CASCADE
estudiante	miembro_grupo	id_estudiante	1:N	CASCADE / CASCADE

Tabla padre	Tabla hija	Clave foránea	Cardinalidad	Acción
grupo	miembro_grupo	id_grupo	1:N	CASCADE / CASCADE
grupo	evento	id_grupo	1:N	CASCADE / CASCADE
estudiante	asistente_evento	id_estudiante	1:N	CASCADE / CASCADE
evento	asistente_evento	id_evento	1:N	CASCADE / CASCADE
estudiante	publicacion	id_estudiante	1:N	CASCADE / CASCADE
publicacion	comentario	id_publicacion	1:N	CASCADE / CASCADE
estudiante	comentario	id_estudiante	1:N	CASCADE / CASCADE
publicacion	reaccion	id_publicacion	1:N	CASCADE / CASCADE
estudiante	reaccion	id_estudiante	1:N	CASCADE / CASCADE

Justificación de ON UPDATE CASCADE. Si el valor de una clave primaria cambia, el cambio se propaga automáticamente a todas las tablas hijas y no quedan referencias rotas.

Justificación de ON DELETE CASCADE por tipo de tabla

- **Dependencia fuerte (perfil, publicacion, comentario, reaccion, evento).** Estas filas no tienen sentido sin su padre: un perfil sin estudiante, un comentario sin publicación o un evento sin grupo serían registros huérfanos.
- **Tablas intermedias (estudiante_habilidad, estudiante_interes, miembro_grupo, asistente_evento, seguir).** Cada fila es solo un vínculo entre dos entidades; si desaparece una de ellas, el vínculo deja de existir.

- **mensaje.** Al eliminar un estudiante se borran los mensajes que envió y recibió. Es una decisión consciente para no dejar mensajes sin emisor o receptor; como alternativa se podría usar SET NULL (permitiendo nulos) o RESTRICT para conservar el historial.

Se descartó RESTRICT / NO ACTION porque obligaría a borrar manualmente todos los datos dependientes antes de eliminar un estudiante, un grupo o una publicación. Se descartó SET NULL porque casi todas las FK son NOT NULL, ya que la relación es obligatoria.

5.Script DDL y conclusiones

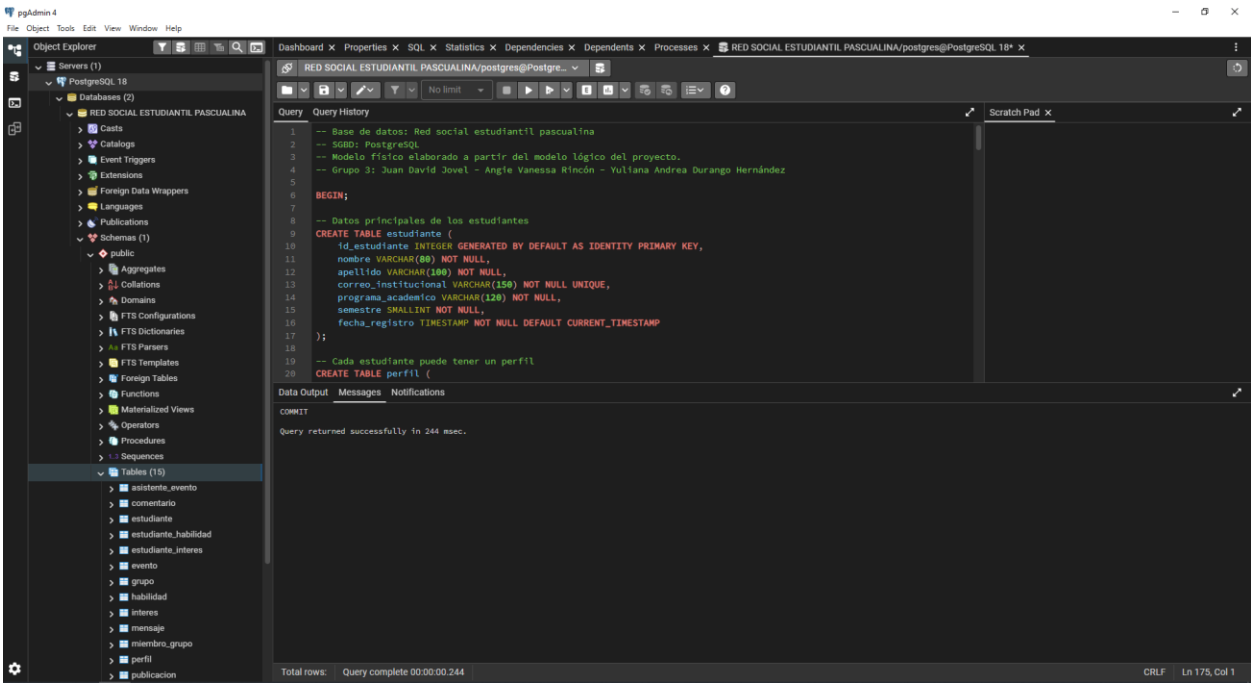
Estructura del archivo modelo_fisico.sql

El script está escrito para PostgreSQL y organizado así:

- **Encabezado:** comentarios con el nombre de la base de datos, el SGBD y los integrantes del grupo.
- **Transacción:** todo el script está entre BEGIN; y COMMIT;, de modo que si alguna sentencia falla no se crea un esquema parcial.
- **Orden de creación:** primero las tablas padre (estudiante, habilidad, interes, grupo) y luego las que dependen de ellas (perfil, tablas intermedias, mensaje, evento, publicacion, comentario, reaccion), para que cada clave foránea referencie una tabla ya existente.
- **Comentarios --** antes de cada bloque, que explican el propósito de las tablas.
- **Restricciones declaradas en línea:** PK, FK con acciones referenciales, UNIQUE, NOT NULL, DEFAULT y CHECK.

Evidencias de la ejecución

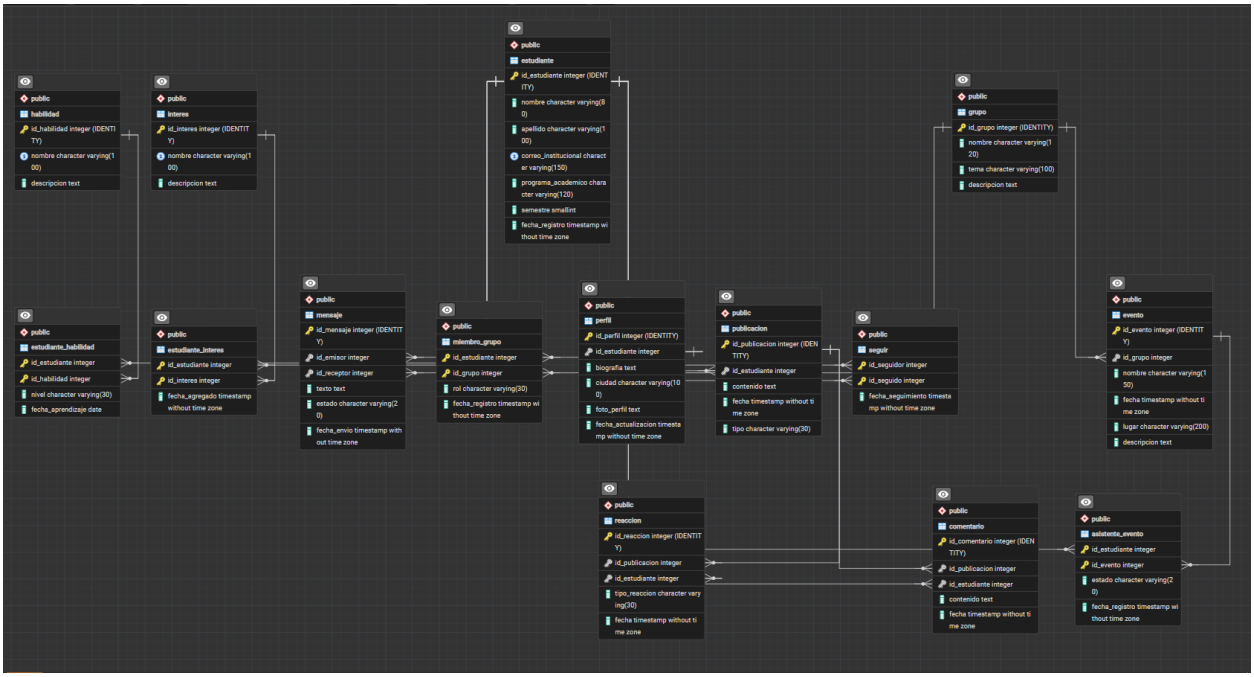
- Ejecución exitosa del script en la herramienta de consulta (mensaje «Query returned successfully»).



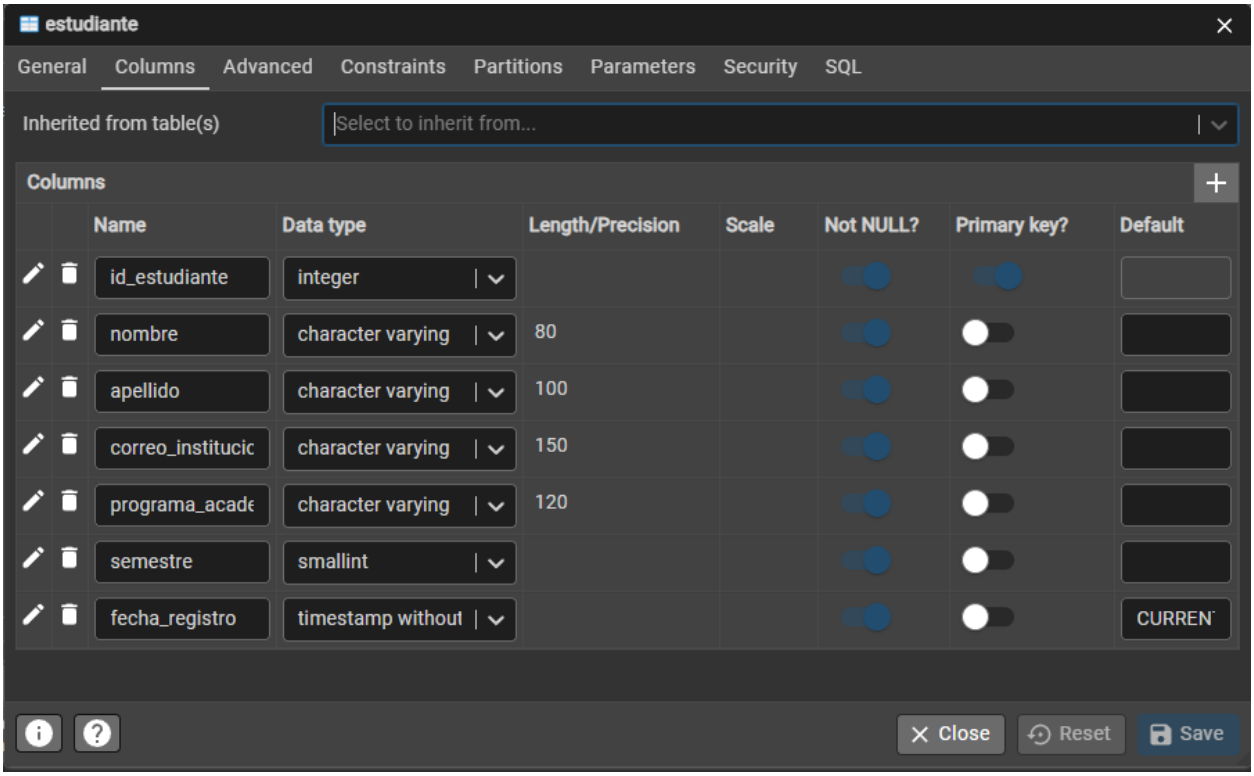
- Árbol de objetos con las 15 tablas creadas.



- Diagrama entidad-relación generado por pgAdmin (*ERD*).



- Estructura ejemplo de la tabla ESTUDIANTE



Conclusiones individuales

Angie Vanessa Rincón

Al comenzar este reto pensaba que pasar del modelo lógico al físico era solo “copiar” el diagrama a código, pero me di cuenta de que cada decisión cuenta. Elegir cuánto espacio ocupa un texto, si un dato es obligatorio o si no se puede repetir cambia la forma en que la base de datos protege la información. Entender que cuando dos cosas se relacionan de muchos a muchos hace falta una tabla aparte para unirlas fue lo que me hizo ver el modelo con más claridad.

Lo que más me hizo pensar fue qué pasa cuando se borra o se cambia un dato. Al principio me parecía que borrar todo lo relacionado era la solución para todo, pero al analizar la tabla de mensajes entendí que eliminar a un estudiante también borra los

mensajes que recibieron otras personas, y que cada decisión tiene un costo. Aprendí que diseñar una base de datos es pensar en lo que pasará cuando la información cambie o desaparezca, y no solo en cómo guardarla.

Trabajar en equipo con Git y pgAdmin 4 me enseñó a documentar y a explicar mis decisiones para que otros las entiendan. Me llevo una base más sólida para los próximos proyectos, y la certeza de que un buen diseño desde el inicio evita muchos problemas después.

Juan David Jovel López

Durante el desarrollo de este trabajo pude comprender mejor cómo se lleva un modelo lógico a una base de datos física utilizando PostgreSQL. Aprendí que no basta con identificar las tablas y sus atributos, sino que también es necesario seleccionar correctamente los tipos de datos, definir las claves primarias y foráneas, y establecer las relaciones entre las tablas para mantener la información organizada y evitar inconsistencias.

Uno de los aspectos que me pareció más importante fue entender cómo funcionan las relaciones entre las entidades, especialmente las relaciones muchos a muchos, que requieren tablas intermedias, y las relaciones entre estudiantes, como el seguimiento dentro de la red social. También comprendí la importancia de las restricciones y de las acciones referenciales para mantener la integridad de los datos.

En el proceso surgieron dudas sobre la elección de algunos tipos de datos y sobre la manera correcta de implementar ciertas relaciones del modelo lógico en el script DDL. Sin embargo, revisar estos aspectos me permitió reconocer que cada decisión en el diseño de una base de datos tiene una función y puede afectar su funcionamiento. En conclusión, este trabajo me ayudó a fortalecer mis conocimientos sobre la creación de tablas y la implementación de relaciones en PostgreSQL, y me permitió entender mejor la importancia de planificar y revisar el diseño antes de implementar una base de datos.